

Date: June 4, 2026

Target Environment: Local Windows Machine (MobaXterm / POSIX Emulation)

Target AWS Infrastructure: Amazon EKS Cluster

Deployment Region: Mumbai, India (ap-south-1) — Optimized for Dhaka network proximity.

1. Prerequisites & Host Configuration

Before starting, ensure your local environment is authenticated with AWS. The installation commands assume your AWS CLI is already set up.

```
# Verify AWS CLI installation and credentials
```

```
aws --version
```

```
aws configure
```

2. Environment Setup & Tool Installation

The Challenge

Initially, an attempt was made to use macOS/Linux package managers (brew) and Windows-native package managers (winget, choco) inside the MobaXterm POSIX emulator environment. This resulted in environment mismatch errors (command not found).

The Solution (Manual Binary Orchestration)

To bypass local package manager limitations inside MobaXterm, we pulled the compiled architecture-specific binary directly from the official repository distribution.

```
# 1. Download the latest compiled Windows AMD64 binary for eksctl
```

```
curl --silent --location "https://github.com/eksctl-  
io/eksctl/releases/latest/download/eksctl_Windows_amd64.zip" -o "eksctl.zip"
```

2. Extract the executable archive

unzip eksctl.zip

3. Clean up the installation zip package

rm eksctl.zip

4. Move the executable to your local bin path for global execution

mv eksctl.exe /bin/

5. Verify installation success (Should return version 0.227.0 or newer)

eksctl version

3. Infrastructure Deployment (EKS Provisioning)

Using eksctl, a declarative command was issued to provision a production-ready, managed Kubernetes environment in the **Mumbai (ap-south-1)** region.

**eksctl create cluster **

**--name mumbai-eks-cluster **

**--region ap-south-1 **

**--nodegroup-name standard-workers **

**--node-type t3.medium **

**--nodes 2 **

--managed

Or,

```
eksctl create cluster --name mumbai-eks-cluster --region ap-south-1 --  
nodegroup-name standard-workers --node-type t3.medium --nodes 2 --  
managed
```

Underlying AWS CloudFormation Automation

When this command executed, eksctl automated the creation of multiple nested CloudFormation stacks to build out the infrastructure lifecycle:

1. **Network Topology Setup:** Automated provisioning of a dedicated Virtual Private Cloud (VPC) with high-availability public and private subnet pairs across multiple Availability Zones.
2. **Control Plane Provisioning:** Created a highly available, AWS-managed Kubernetes master API plane and etcd state storage.
3. **Managed Node Groups:** Initialized 2 EC2 worker nodes using the Amazon EKS-optimized Linux AMI. These nodes reside in private subnets, shielded from direct internet scanning (which is why direct IP ping actions fail by design).
4. **Local Access Integration:** Automatically populated and updated the local file path (~/.kube/config) to authorize local kubectl requests to the remote API server.
5. **4. Post-Deployment Cluster Verification**
6. Once the automation script outputted a cluster status of ready, configuration verification was completed using the Kubernetes command-line tool (kubectl).

Review the status of the provisioned computing nodes

```
kubectl get nodes
```

Expected Output (as verified):

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-20-148.ap-south-1.compute.internal	Ready	<none>	11m	v1.34.8-eks-3385e9b
ip-192-168-64-117.ap-south-1.compute.internal	Ready	<none>	11m	v1.34.8-eks-3385e9b

5. Next Steps: Workload Deployment

To start running services on the new cluster, deploy a test container framework to ensure scheduling is completely operational:

1. Spin up an Nginx instance

kubectl create deployment nginx-test --image=nginx

2. Monitor the pod deployment status until it reads 'Running'

kubectl get pods

3. Inspect system-level add-ons (like the core metrics server)

kubectl get pods -n kube-system

6. Teardown & Lifecycle Management (Cost Prevention)

To prevent unexpected AWS infrastructure bills, always dismantle the cloud formation stacks when work or testing sessions conclude. This automation cleans up the EC2 instances, VPC parameters, and control planes completely:

eksctl delete cluster --name mumbai-eks-cluster --region ap-south-1